❶ Adds supervision to recover from failure

This also preserves all buffered commands and makes them available for the next incarnation of the actor. With this configuration, if the actor crashes, it will attempt to restart after 10 seconds with a random accuracy of 10%. If it cannot be restarted within this time, it will attempt to do so with an increasing delay of 20, 30, 40, and 50 seconds, up to a maximum of 60 seconds.

9.3.2 Recovery

When an actor starts or is restarted, it always receives a recovery status signal, regardless of whether it has entries to read in the journal. After the actor has finished reading the events available in the journal, it receives the signal. This is useful, for example, if you want to send a message to another actor to let it know that this actor is ready to process data, or if you want to publish this signal so that it can be monitored by application operators.

As with any other signal, add `receiveSignal`, which gets two parameters (the current status and the message):

```
EventSourcedBehavior[Command, Event, State](
  persistenceId = ...,
  emptyState = ...,
  commandHandler = ...,
  eventHandler = ...,
).receiveSignal {
    case (state, RecoveryCompleted) =>        ❶
      ...
    case (state, RecoveryFailed(failure)) =>  ❷
      ...
}
```

❶ Possibility to react after state reload is completed

❷ Possibility to react if state reload can't complete

If the entire journal is read without trouble, the signal received is a `RecoveryCompleted`. If it fails, a `RecoveryFailed(failure: Throwable)` signal is emitted, the cause is logged, and a `JournalFailureException` is thrown that stops or restarts the actor if a `restartWithBackoff` has been set. Some of the most common reasons for failure are that the event is corrupted or becomes corrupted during the transfer, or a new version of the actor is incompatible with an event.

9.3.3 Snapshotting

Restoring the state of the persistent entity by reading each event can take a very long time. The more extensive the actor's history is, the worse its initialization time. To reduce the load time, you can take snapshots when one of two types of conditions is met: an event contains a certain value, or a number of events have been stored in the journal.